Best Goal Search Algorithm

Typeset from Manuscript Notes

October 10, 2026

1 Overview & Time Complexity

`GoalSearch.BestSearch` navigates state transitions from an initial state S_0 towards a goal state S_g. It is an iterative depth-first search over an explicit stack. If S_g is reached, that path is returned. Otherwise, it returns the path to the leaf state with the highest balance among the leaves it visits.

$$\underline{\text{Best Goal Search}} \implies \text{Complexity } O\big(|A|^{\text{MAX-DEPTH}}\big)$$

Where:

- $A = (A_0, \ldots, A_{|A|-1})$: The ordered list of available actions (deposit schemes), indexed by a.
- $|A|$: The branching factor (number of actions).
- MAX-DEPTH: The deepest stack reached before the reachability test fails (bounded by the goal time horizon).

2 Interface

- **Transition**: $(S', c) = T(S, A_a)$ returns the successor state S' and the number of times c that action A_a is repeated to reach it.
- **Reachability test**: $isGR(S', S_g)$ returns one of:
  - `GOAL_REACHED`: S' satisfies the goal.
  - `GOAL_NOT_YET_REACHED`: S' can be extended further.
  - `GOAL_NOT_REACHABLE`: S' overshoots the goal and is discarded.
- **Stack entries**: each entry is $((a, c), S')$: the action index, its repeat count, and the state it produced.

3 Typeset Pseudocode

```
GoalSearch(Sg, A, T, isGR)

BestSearch(S0):
    S = S0, stack = [], a = 0
    best = [], bestBalance = 0
    prev = GOAL_NOT_YET_REACHED

    loop:
        (S', c) = T(S, A[a])
        st = isGR(S', Sg)

        if st == GOAL_NOT_YET_REACHED:      # go one level deeper
            stack.push(((a, c), S'))
            S = S'
            a = 0

        elif st == GOAL_NOT_REACHABLE:      # try the next sibling action
            a = a + 1
            if a >= |A|:                    # all siblings tried: backtrack
                if stack is empty:
                    break
                ((a, _), _) = stack.pop()
                a = a + 1
                if stack is empty:
                    S = S0
                    break
                S = top(stack).state
            elif prev == GOAL_NOT_YET_REACHED:
                # S is a leaf: keep the path if it has the best balance
                if S.balance > bestBalance:
                    bestBalance = S.balance
                    best = copy(stack)

        else:
            stack.push(((a, c), S'))
            best = copy(stack)
            break

        prev = st

    L = copy(best)
    return st

GetFinalList():
    if L is empty:
        return (L, Sg.balance - S.balance)
    return (L, Sg.balance - L[-1].state.balance)

# L has the answer; the second value is the shortfall to the goal
```

Listing 1: Best Goal Search Pseudocode

4 Step-by-Step Logic

1. **Transition**: Apply action A_a to the current state S to get $(S', c) = T(S, A_a)$ and classify S' with $isGR(S', S_g)$.

2. **Descend**: If S' is `GOAL_NOT_YET_REACHED`, push $((a, c), S')$, make S' the current state and restart from the first action $a = 0$.
3. **Leaf Evaluation**: If S' is `GOAL_NOT_REACHABLE` right after a descent, the current state S is a leaf. If its balance beats the best seen so far, record a copy of the stack as the best path.
4. **Next Sibling**: After a `GOAL_NOT_REACHABLE` result, move to the next action $a + 1$ at the same depth.
5. **Backtracking**: When every action at a depth has been tried, pop the top entry and continue from the action after the popped one, with the new stack top as the current state. The search stops once the stack becomes empty.
6. **Goal Reached**: If S' is `GOAL_REACHED`, push $((a, c), S')$, take the whole stack as the answer and stop.
7. **Result**: The stored path L is returned by `GetFinalList` together with the shortfall S_g.balance $-$ S_{end}.balance, where S_{end} is the last state on L, or the initial state when L is empty.